

Query 1

List of all people who have studied at the UPM or UAM.

- Join the users and the ed_center on the educational_centers.email.
- Separate the ed_details into documents.
- Select documents where ed_center name is UPM or UAM.
- Group by user name to avoid duplication.

```
db.User.aggregate([
  {
    $lookup: {
      from: "Educational_Center",
      localField: "educational_centers.email",
      foreignField: "email",
      as: "ed_details",
    },
  },
  { $unwind: "$ed_details" },
  {
    $match: {
      "ed_details.name": {
        $in: [
          "Escuela de Negocios Alicante",
          "Instituto Tecnológico de Granada",
        ],
      },
    },
  },
  { $group: { _id: "$name" } },
]);
```

Version with all the info of the users:

```
db.User.aggregate([
  {
    $lookup: {
      from: "Educational_Center",
      localField: "educational_centers.email",
      foreignField: "email",
      as: "ed_details",
    },
  },
  { $unwind: "$ed_details" },
  {
    $match: {
      "ed_details.name": {
        $in: [
```

```

        "Escuela de Negocios Alicante",
        "Instituto Tecnológico de Granada",
    ],
    },
},
{
  $project: {
    name: 1,
    email: 1,
    address: 1,
    interests: 1,
    skills: 1,
    educational_centers: 1,
    companies: 1,
    description: 1,
  },
},
]);

```

Query 2

Different universities where people residing in Madrid have studied.

- Filter users who live in Madrid.
- Join with Educational_Center using each center's email.
- Unwind joined ed_details into separate docs.
- Group by educational center name and return distinct names.

```

db.User.aggregate([
  { $match: { address: { $regex: "Madrid" } } },
  {
    $lookup: {
      from: "Educational_Center",
      localField: "educational_centers.email",
      foreignField: "email",
      as: "ed_details",
    },
  },
  { $unwind: "$ed_details" },
  { $group: { _id: "$ed_details.name" } },
]);

```

Query 3

People whose profile description includes “Big Data” or “Artificial Intelligence”.

- Use \$match with \$or to find descriptions containing the terms.
- Use case-insensitive regex to catch variations of casing.

```
db.User.aggregate([
  {
    $match: {
      $or: [
        { description: { $regex: "Big Data", $options: "i" } },
        { description: { $regex: "Artificial Intelligence", $options: "i" } },
      ],
    },
  },
]);
```

Query 4

Save users who completed any study in 2017 or later into a new collection.

- Match users where at least one educational_centers entry has a completion_date >= 2017.
- Output results to a new collection using \$out.

Note: If completion_date is stored as a string year, the first pipeline works. If stored as a date, use the ISODate variant.

String/year version:

```
db.User.aggregate([
  { $match: { "educational_centers.completion_date": { $gte: "2017" } } },
  { $out: "users_completed_studies_in_2017_or_after" },
]);
```

Date-version (completion_date is a Date):

```
db.User.aggregate([
  {
    $match: {
      "educational_centers.completion_date": { $gte: ISODate("2017-01-01") },
    },
  },
  { $out: "users_completed_studies_in_2017_or_after" },
]);
```

Query 5

Average number of studies for people who have worked at Microsoft.

- Join users with Company collection via companies (company emails).
- Unwind company details and match company name to Microsoft.
- Group and compute the average number of educational_centers per user.

```
db.User.aggregate([
  {
    $lookup: {
      from: "Company",
      localField: "companies",
      foreignField: "email",
      as: "c_details",
    },
  },
  { $unwind: "$c_details" },
  { $match: { "c_details.name": { $regex: "Microsoft", $options: "i" } } },
  {
    $group: {
      _id: "microsoft_workers",
      average_studies: { $avg: { $size: "$educational_centers" } },
    },
  },
]);
```

Query 6

Average geodesic distance to work for current Google workers.

- Use \$geoNear as the first stage to compute distance from a Google office coordinate (replace with the exact office coords).
- Join with Company collection, unwind and match company name to Google.
- Group and compute the average distance (distance stored in meters).

Replace coordinates with the correct Google office location if needed.

```
db.User.aggregate([
  {
    $geoNear: {
      near: { type: "Point", coordinates: [-73.989308, 40.741895] },
      distanceField: "distance_from_google",
      spherical: true,
    },
  },
  {
    $lookup: {
```

```

        from: "Company",
        localField: "companies",
        foreignField: "email",
        as: "c_details",
    },
},
{ $unwind: "$c_details" },
{ $match: { "c_details.name": { $regex: "Google", $options: "i" } } },
{
    $group: {
        _id: "google_workers",
        average_distance_meters: { $avg: "$distance_from_google" },
    },
},
]);

```

Query 7

Top 3 universities that most often appear as study centers.

- Join users with Educational_Center on educational_centers.email.
- Unwind ed_details and group by ed_details.name counting occurrences.
- Sort by count descending and limit to the top three.

```

db.User.aggregate([
{
    $lookup: {
        from: "Educational_Center",
        localField: "educational_centers.email",
        foreignField: "email",
        as: "ed_details",
    },
},
{ $unwind: "$ed_details" },
{ $group: { _id: "$ed_details.name", nb_users: { $sum: 1 } } },
{ $sort: { nb_users: -1 } },
{ $limit: 3 },
]);

```